6

Communication and Interaction

In Chapter 2, we saw that communication in multi-agent systems is typically based on the *speech act* paradigm, and we briefly reviewed the KQML and FIPA agent communication languages. In this chapter, we see how agent communication is accomplished in ***Jason***.

Recall that in Section 4.1 we saw that, at the beginning of each reasoning cycle, agents check for messages they might have received from other agents. Any message received[1] by the checkMail method (see Figure 4.1) has the following structure:

$$\langle sender, illoc\_force, content \rangle$$

where *sender* is the AgentSpeak atom (i.e. a simple term) which is the name by which the agent who sent the message is identified in the multi-agent system; *illoc_force* is the *illocutionary force*, often called *performative*, which denotes the intention of the sender; and *content* is an AgentSpeak term which varies according to the illocutionary force. In this chapter, we see each of the available performatives and how they are interpreted by ***Jason***.

While ***Jason*** interprets automatically any received messages according to the formal semantics given in [100], it is the programmer's responsibility to write plans which make the agent take appropriate communicative action. Ideally, future work on agent communication and agent programming languages should lead to the development of tools that allow high-level specifications of agent interactions (i.e. protocols) to be automatically transformed into plan skeletons which ensure the agent is capable of following the relevant protocols.

[1]Note that the format of the message from the point of view of the sending and receiving agents is slightly different; for example, a message to be sent has a reference to the receiver whereas a received message has a reference to the sender. The format explained here is that from the point of view of the *receiver* specifically.

6.1 Available Performatives

Messages are sent, as we saw in Chapter 3, with the use (in plan bodies) of a special pre-defined internal action that is used for inter-agent communication. A list of all pre-defined internal actions is presented in Appendix A.3; because this one is special in that it is the basis for agent interaction in ***Jason*** – and interaction in multi-agent systems is an essential issue – we need a detailed account of the internal action and all possible uses of it so that programmers can make effective use of agent communication in ***Jason***; this is what we do in this chapter. The general form of the pre-defined internal action for communication is:

```
.send(receiver,illocutionary_force,propositional_content)
```

where each parameter is as follows. The `receiver` is simply referred to using the name given to agents in the multi-agent system configuration file. When there are multiple instances of an agent, ***Jason*** creates individual names by adding numbers starting from 1. For example, if there are three instances of agent `a`, ***Jason*** creates agents `a1`, `a2` and `a3`. The `receiver` can also be a list of agent names; this can be used to multicast the message to the group of agents in the list. The `propositional_content` is a term often representing a literal but at times representing a triggering event, a plan or a list of either literals or plans. The `illocutionary_force`s available are briefly explained below; the name of the performatives and the informal semantics are similar to KQML [61, 69]. The next section will show in detail the changes in an agent's mental state that occur when messages are received. The performatives with suffix '`How`' are similar to their counterparts but related to plans rather than predicates.

The list of all available performatives are as follows, where *s* denotes the sender and *r* the receiver:

`tell`: *s* intends *r* to believe (that *s* believes) the literal in the message's content to be true;

`untell`: *s* intends *r* not to believe (that *s* believes) the literal in the message's content to be true;

`achieve`: *s* requests *r* to try and achieve a state of affairs where the literal in the message content is true (i.e. *s* is delegating a goal to *r*);

`unachieve`: *s* requests *r* to drop the goal of achieving a state of affairs where the message content is true;

`askOne`: *s* wants to know if the content of the message is true for *r* (i.e. if there is an answer that makes the content a logical consequence of *r*'s belief base, by appropriate substitution of variables);

askAll: *s* wants all of *r*'s answers to a question;

tellHow: *s* informs *r* of a plan (*s*'s know-how);

untellHow: *s* requests that *r* disregard a certain plan (i.e. delete that plan from its plan library);

askHow: *s* wants all of *r*'s plans that are relevant for the triggering event in the message content.

Agents can ignore messages, for example if a message delegating a goal is received from an agent that does not have the power to do so, or from an agent towards which the receiver has no reason to be generous. Whenever a message is received by an agent, if the agent accepts the message, an event is generated which is handled by pre-defined AgentSpeak plans (shown in the next section).

There is another internal action for communication used to broadcast messages rather than send a message to a particular (or a group) of agents. The usage is `.broadcast(illocutionary_force,propositional_content)`, where the parameters are as above, but the message is sent to all agents registered in the agent society (i.e. the ***Jason*** multi-agent system; note that agents can be created and destroyed dynamically).

6.2 Informal Semantics of Receiving Messages

The effects on a ***Jason*** agent of receiving a message with each of these types of illocutionary acts is explained (formally) in Section 10.3, following the work in [70, 100]. We here explain this informally, considering the following example. Assume agent `s` (the 'sender') has just executed the following formula that appeared in one of its intentions:

```
.send(r,tell,open(left_door));
```

agent `r` would then receive the message

$$\langle \texttt{s}, \texttt{tell}, \texttt{open(left\_door)} \rangle.$$

The first thing that happens when this message is taken from `r`'s mailbox is to pass it as argument in a call to the SocAcc method (it stands for 'socially acceptable'; see Section 7.2). This method can be overridden for each agent to specify which other agents are authorised to send which types of messages. If SocAcc returns false, the message is simply discarded. If the message is accepted, it will be processed by agent `r` as explained below; the list gives all possible combinations of illocutionary forces and content types. Even though we are explaining what happens when the message is received by `r`, it makes the presentation easier to use the internal action used by `s` to *send* the message rather than the message in the format received by `r`.

We shall now consider each type of message that could be sent by `s`. We organise the examples in the following categories: information exchange, goal delegation, information seeking and know-how related communication.

Information Exchange

`.send(r, tell, open(left_door)):` the belief `open(left_door)[source(s)]` will be added to `r`'s belief base. Recall that this generally means that agent `r` believes the left door is open because agent `s` said so, which normally means that `s` itself believed the left door was open at the time the message was sent and wanted `r` to believe so too. When programming agents that need to be more conscious of other agents' trustworthiness, the plans need to be programmed so that the semantics of `open(left_door)[source(s)]` is simply that, at the time the message was received, agent `s` wanted agent `r` to believe `open(left_door)`, rather than either of the agents actually believing it.

`.send([r1,r2], tell, open(left_door)):` the belief `open(left_door)[source(s)]` will be added to the belief base of both `r1` and `r2`.

`.send(r, tell, [open(left_door),open(right_door)]):` the belief `open(left_door)[source(s)]` and the belief `open(right_door)[source(s)]` will both be added to `r`'s belief base.

These variations using a list of recipients of having a list in the content field can also be used with all other types of messages. However, note that for a synchronous `ask` message, the first answer from whatever agent makes the suspended intention be resumed.

`.send(r,untell,open(left_door)):` the belief `open(left_door)[source(s)]` will be removed from `r`'s belief base. Note that if agent `r` currently believes `open(left_door)[source(t), source(s), source(percept)]`, meaning the agent perceived, and was also told by agents `s` and `t`, that the left door is open, the belief base will be updated to have `open(left_door)[source(t), source(percept)]` instead. That is, only the fact that `s` wanted `r` to believe `open(left_door)` is removed from the belief base.

Figure 6.1 shows a summary of the changes that take place within agents due to communication of type 'information exchange'.

Cycle	*s* actions	*r* belief base	*r* events
1	`.send(r,`*`tell`*`, open(left_door))`		
2		`open(left_door) [source(s)]`	`+open(left_door) [source(s)]`
3	`.send(r,`*`untell`*`, open(left_door))`	`open(left_door) [source(s)]`	
4			`-open(left_door) [source(s)]`

Figure 6.1 Changes in the receiver's state for (un)tell messages.

Goal Delegation

`.send(r, achieve, open(left_door))`: the event $\langle$`+!`*open*(*left_door*)[*source*(*s*)], $\top\rangle$ is created and added to `r`'s set of events. This means that, if the agent has plans that are relevant for when it needs to achieve the state of affairs where the door is open, and one such plan happens to be applicable when the event is selected, that plan will become intended. This in turn means that the agent commits to see to it that the door gets opened. Note how the different illocutionary forces make a significant difference on how the message is interpreted by the agent, as compared with the cases above. Interestingly, ***Jason*** adds an annotation that the goal was delegated by agent `s`, so this can be used in the plans to decided whether to actually take any action or not, and to decide what is the most appropriate course of action as well (note that this is on top of the message being socially acceptable).

`.send(r, unachieve, open(left_door))`: the internal action `.drop_desire(open(left_door))` is executed. This internal action is described in Section A.3; it causes current and potential intentions on instances of a goal `open(left_door)` to be dropped, without generating a plan failure event for the plans that required that goal to be achieved. The default communication plan for this performative could be overridden, for example to use `.fail_goal`, but then care must be taken to avoid the goal being attempted again within plan failure handling (in case the agent is to follow the request of dropping the goal).

Cycle	*s* actions	*r* intentions	*r* events
1	`.send(r,`*`achieve`*`, open(left_door))`		
2			`+!open(left_door) [source(s)]`
3		`!open(left_door) [source(s)]`	
4	`.send(r,`*`unachieve`*`, open(left_door))`	`!open(left_door) [source(s)]`	
5			

Figure 6.2 Changes in the receiver's state for (un)achieve messages.

Advanced

Note that, in the standard interpretation, *all* instances of that goal currently intended by the agent will be dropped. In some applications, it might be useful to only drop the instances of the goal that were delegated by the sender itself. This can be done easily using the `source` annotations created by ***Jason*** and by creating a special plan to interpret `unachieve` messages; we explain later how to create specialised plans for interpreting received messages.
A common situation will be for an agent to ask another agent to drop a goal that itself delegated to that agent in the past. In that case, the goal-addition triggering event appears in the plan just above the pre-defined plan for handling received messages. Although in this case usually nothing needs to be done before the goal is dropped, if necessary in a particular application, there is a chance for a plan to be executed if anything needs to be done before the goal is dropped, by creating a contingency plan for the pre-defined plan used to interpret `unachieve` messages.

Figure 6.2 shows a summary of the changes that take place within agents due to communication of type 'goal delegation'.

Information Seeking

`.send(r, askOne, open(Door), Reply)`: `s`'s intention that executed this internal action will be suspended until a reply from `r` is received. On `r`'s side, the message ⟨`s`, `askOne`, `open(Door)`⟩ is received and (if accepted) the ***Jason*** predefined plan to interpret `askOne` messages itself sends the appropriate reply; for example, a reply with

content `open(left_door)` if that was in `r`'s belief base at the time. This is done with a test goal, so if the relevant information is not in the belief base the agent can also try to execute a plan to obtain the information, if such plan exists in the plan library. When the reply is received by `s`, the content in that message is instantiated to the `Reply` variable in the internal action executed by `s`. The intention is then resumed, hence the formula after the `.send` internal action is only executed after `s` has received a reply from `r`. Interestingly, this means that an `askOne` message is similar to a test goal but in another agent's belief base (if the other agent so permits). Further, as mentioned earlier, recall that if a synchronous `ask` message is multi-cast, the first reply received from any of the addressees of the original message is used to resume the suspended intention.

Note that variable `Door` in this internal action would *not* be instantiated after the action is executed, only the *Reply* variable is. To obtain a similar effect of instantiating the `Door` variable as if the reply was obtained from the message content itself (the third parameter), the action needs to be written as '`.send(r,askOne,open(Door),open(Door))`'. This looks redundant but is necessary for technical reasons, e.g. to allow for negative replies and for communication with agents that are not written in AgentSpeak: the content and reply can be any AgentSpeak term, including a string, which is then general enough to allow communication among heterogeneous agents.

If the content of `s`'s ask message does not unify with any of `r`'s beliefs, variable `Reply` would be unified with the reserved keyword `false`. So, to write a plan that can only carry on executing if another agent does *not* believe `open(Door)` (i.e. that it does not believe that any door is open), we include '`.send(r,ask,open(Door),false);`' in the plan body. In this case, if `r` believes the left door is open, the reply would be `open(left_door)`, which cannot be unified with `false`, therefore the internal action `.send` fails, and so does the plan where the action appeared. Note that checking whether the receiver believes an instance of the propositional content uses a *test goal*, therefore before a `false` answer is sent, an event `+?open(Door)` would be generated, and if the agent had a relevant (and applicable) plan, that plan would be executed and could then lead to the answer to be sent to the agent waiting for a reply to an `ask` message.

One frequent mistake is to think that, because '`ask`' messages are synchronous, agents become idle waiting for any reply. Recall that an AgentSpeak agent typically has multiple foci of attention (i.e. multiple

intentions in the set of intentions) and so the agent can carry on doing work for the intentions that do not need to wait for an answer. Also, asynchronous requests for information can be done by providing the receiver with a plan to achieve the goal of providing the required information, and the sender can have a plan which simply sends an `achieve` message with that particular goal as message content.

Advanced

While there is no problem with the agent becoming completely idle, there is, however, a problem in that agents could end up with many suspended intentions which, for example, could be dropped as they will never be successfully completed (e.g. because the relevant agent left the society, or failed to receive the message). To help alleviate the problem, any 'ask' message can have an optional fifth parameter which determines a 'timeout' for the wait for a reply (in milliseconds), for example as in '`.send(r,askOne,open(Door),Reply,3000)`' to wait for a reply within 3 seconds. If the request for information times out, the `Reply` variable will be instantiated with the term `timeout`, so that programmers will know that no reply was obtained; this can be used by programmers in a contingency plan to, for example, 'give up' by dropping the intention, or 'insist' by sending the message again.

`.send(r, askOne, open(Door))`: as above but asynchronous, in the sense that the `askOne` message is sent to `r` but the intention is not suspended to wait for a reply; this means that the formula in the plan body immediately after the `.send` internal action could in principle execute in the next reasoning cycle (if selected for execution). Of course this should be used when the remainder of the plan does not depend on the reply being received to finish executing. When a reply is received, a new belief is likely to be added which, recall, generates an event. When further action is necessary if a reply eventually arrives, programmers then typically write plans to deal, in a separate intention, with the event generated by the received response.

`.send(r, askAll, open(Door), Reply)`: as for `askOne` (the synchronous version) but `Reply` is a list with all possible answers that `r` has for the content. For example, if at the time of processing the `askAll` message agent `r` believed both `open(left_door)` and `open(right_door)`, variable `Reply` in the internal action would be instantiated with the list '`[open(left_door),open(right_door)]`'.

An empty list as reply denotes `r` did not believe anything that unifies with `open(Door)`.

Figure 6.3 shows a summary of the changes that take place within agents due to communication of type 'information seeking'. Note that this figure is for the

***r* belief base**
`open(left_door)`
`open(right_door)`

Cycle	*s* actions / unifier / events	*r* actions
1	`.send(r,`*`askOne`*`,` `open(Door),Reply)`	
2		`.send(s,tell,open(left_door))`
3	`Reply` ↦ `open(left_door)`	
4	`.send(r,`*`askOne`*`,` `closed(Door),Reply)`	
5		`.send(s,tell,false)`
6	`Reply` ↦ `false`	
7	`.send(r,`*`askOne`*`, open(Door))`	
8		`.send(s,tell,open(left_door))`
9	`+open(left_door)[source(s)]`	
10	`.send(r,`*`askAll`*`,` `open(D),Reply)`	

11		`.send(s,tell,`
		`[open(left_door),`
		`open(right_door)])`

12	`Reply` ↦ `[open(left_door),` `open(right_door)]`	
13	`.send(r,`*`askAll`*`, open(D))`	

14		`.send(s,tell,`
		`[open(left_door),`
		`open(right_door)])`

15	`+open(left_door)[source(s)]` `+open(right_door)[source(s)]`	

Figure 6.3 Examples of ask protocols.

default `ask` protocol assuming no relevant plans with triggering events `+?` are available in the receiver agent.

Know-how Related

`.send(r, tellHow, "@pOD +!open(Door) : not locked(Door) <- turn_handle(Door); push(Door); ?open(Door).");`: the string in the message content will be parsed into an AgentSpeak plan and added to `r`'s plan library. Note that a string representation for a plan in the agent's plan library can be obtained with the `.plan_label` internal action given the label that identifies the required plan; see page 243 for details on the `.plan_label` internal action.

`.send(r, tellHow, ["+!open(Door) : locked(Door) <- unlock(Door); !!open(Door).", "+!open(Door) : not locked(Door) <- turn_handle(Door); push(Door); ?open(Door)."]);`: each member of the list must be a string that can be parsed into a plan; all those plans are added to `r`'s plan library.

`.send(r, untellHow, "@pOD");`: the string in the message content will be parsed into an AgentSpeak plan label; if a plan with that label is currently in `r`'s plan library, it is removed from there. Make sure a plan label is explicitly added in a plan sent with `tellHow` if there is a chance the agent might later need to tell other agents to withdraw that plan from their belief base.

`.send(r, askHow, "+!open(Door)");`: the string in the message content must be such that it can be parsed into a triggering event. Agent `r` will reply with a list of all *relevant plans* it currently has for that particular triggering event. In this example, `s` wants `r` to pass on his know-how on how to achieve a state of affairs where something is opened (presumably doors, in this example). Note that there is not a fourth argument with a 'reply' in this case, differently from the other 'ask' messages. The plans sent by `r` (if any) will be automatically added to `s`'s plan library. In Section 11.2, we explain a more elaborate approach to plan exchange among collaborating agents.

Advanced

The AgentSpeak Plans for Handling Communication

As we mentioned earlier, instead of changing the interpreter to allow for the interpretation of received speech-act based messages, we have written AgentSpeak plans to interpret received messages; ***Jason*** includes such plans at

the end of the plan library when any agent starts running. There are separate plans to handle each type of illocutionary force, and in some cases to handle different situations (e.g. whether the agent has an answer for an 'ask' message or not). The advantage of the approach of using plans to interpret messages is that the normal event/intention selection functions can be used to give appropriate priority to communication requests, and also this allows users to customise the way messages are interpreted easily, by just including in their AgentSpeak code new versions of the pre-defined communication plans. Because the user-defined plans will appear first in the plan library, they effectively override the pre-defined ones. The predefined plans are shown below, and they can also be found in file `src/asl/kqmlPlans.asl` of ***Jason***'s distribution.

By now, readers will be able to understand the plans, as they are rather simple, so we do not provide detailed descriptions. One interesting thing to note, though, is the heavy use of higher-order variables – variables instantiated with a belief, goal, etc. rather than a term (see the use of formulæ such as `+CA` and `!CA` in the plans below). Another noticeable feature of the plans is the use of various internal actions: as some of these have not yet been introduced, readers aiming to understand these plans in detail might need to refer back an forth to Appendix A.3, where we describe in detail all the predefined internal actions in ***Jason*** (even though the action names will give a good intuition of what they do). Also, note that the names of variables used are long and unusual to ensure they do not conflict with user variables in the message content.

```
// Default plans to handle KQML performatives
// Users can override them in their own AS code
//
// Variables:
//   KQML_Sender_Var:  the sender (an atom)
//   KQML_Content_Var: content (typically a literal)
//   KQML_MsgId:       message id (an atom)
//


/* ---- tell performatives ---- */

@kqmlReceivedTellStructure
+!kqml_received(KQML_Sender_Var, tell, KQML_Content_Var, KQML_MsgId)
   :  .structure(KQML_Content_Var) &
      .ground(KQML_Content_Var) &
      not .list(KQML_Content_Var)
   <- .add_annot(KQML_Content_Var, source(KQML_Sender_Var), CA);
      +CA.
@kqmlReceivedTellList
+!kqml_received(KQML_Sender_Var, tell, KQML_Content_Var, KQML_MsgId)
   :  .list(KQML_Content_Var)
   <- !add_all_kqml_received(KQML_Sender_Var,KQML_Content_Var).
```

```
@kqmlReceivedTellList1
+!add_all_kqml_received(_,[]).

@kqmlReceivedTellList2
+!add_all_kqml_received(S,[H|T])
   :  .structure(H) &
      .ground(H)
   <- .add_annot(H, source(S), CA);
      +CA;
      !add_all_kqml_received(S,T).

@kqmlReceivedTellList3
+!add_all_kqml_received(S,[_|T])
   <- !add_all_kqml_received(S,T).

@kqmlReceivedUnTell
+!kqml_received(KQML_Sender_Var, untell, KQML_Content_Var, KQML_MsgId)
   <- .add_annot(KQML_Content_Var, source(KQML_Sender_Var), CA);
      -CA.


/* ---- achieve performatives ---- */

@kqmlReceivedAchieve
+!kqml_received(KQML_Sender_Var, achieve, KQML_Content_Var, KQML_MsgId)
   <- .add_annot(KQML_Content_Var, source(KQML_Sender_Var), CA);
      !!CA.

@kqmlReceivedUnAchieve[atomic]
+!kqml_received(KQML_Sender_Var, unachieve, KQML_Content_Var, KQML_MsgId)
   <- .drop_desire(KQML_Content_Var).


/* ---- ask performatives ---- */

@kqmlReceivedAskOne1
+!kqml_received(KQML_Sender_Var, askOne, KQML_Content_Var, KQML_MsgId)
   <- ?KQML_Content_Var;
      .send(KQML_Sender_Var, tell, KQML_Content_Var, KQML_MsgId).

@kqmlReceivedAskOne2 // error in askOne, send untell
-!kqml_received(KQML_Sender_Var, askOne, KQML_Content_Var, KQML_MsgId)
   <- .send(KQML_Sender_Var, untell, KQML_Content_Var, KQML_MsgId).

@kqmlReceivedAskAll1
+!kqml_received(KQML_Sender_Var, askAll, KQML_Content_Var, KQML_MsgId)
   :  not KQML_Content_Var
   <- .send(KQML_Sender_Var, untell, KQML_Content_Var, KQML_MsgId).

@kqmlReceivedAskAll2
+!kqml_received(KQML_Sender_Var, askAll, KQML_Content_Var, KQML_MsgId)
   <- .findall(KQML_Content_Var, KQML_Content_Var, List);
      .send(KQML_Sender_Var, tell, List, KQML_MsgId).
```

```
/* ---- know-how performatives ---- */

// In tellHow, content must be a string representation
// of the plan (or a list of such strings)

@kqmlReceivedTellHow
+!kqml_received(KQML_Sender_Var, tellHow, KQML_Content_Var, KQML_MsgId)
   <- .add_plan(KQML_Content_Var, KQML_Sender_Var).

// In untellHow, content must be a plan's
// label (or a list of labels)
@kqmlReceivedUnTellHow
+!kqml_received(KQML_Sender_Var, untellHow, KQML_Content_Var, KQML_MsgId)
   <- .remove_plan(KQML_Content_Var, KQML_Sender_Var).

// In askHow, content must be a string representing
// the triggering event
@kqmlReceivedAskHow
+!kqml_received(KQML_Sender_Var, askHow, KQML_Content_Var, KQML_MsgId)
   <- .relevant_plans(KQML_Content_Var, ListAsString);
      .send(KQML_Sender_Var, tellHow, ListAsString, KQML_MsgId).
```

The plans above are loaded by ***Jason*** at the end of any AgentSpeak program; users can customise the interpretation of received speech-act based messages by creating similar plans (which can appear anywhere in the program[2]). Whilst it can be useful to customise these plans for very specific applications, much care should be taken in doing so, to avoid changing the semantics of communication itself. If a programmer chooses not to conform to the semantics of communication as determined by the pre-defined plans (and the literature referred to earlier), sufficient documentation must be provided. Before doing so, it is important to ensure it is necessary not to conform to the given semantics; there are often ways of getting around specific requirements for communication without changing the semantics of communication. An example of a situation where simple customisation can be useful is as follows. Suppose an agent needs to remember that another agent asked it to drop a particular goal. Plan `@kqmlReceivedUnAchieve` could be changed so that the plan body includes `+askedToDrop(KQMLcontentVar)[source (KQML_Sender_Var)]`, possibly even with another annotation stating when that occurred. In other cases, it might be useful to change that plan so that an agent *a* can only ask another agent *b* to drop a goal

[2]This assumes the applicable plan selection function has not been overridden, so applicable plans are selected in the order they appear in the program.

that was delegated by *a* itself. This could be easily done by adding this plan to the source code for agent *b*:

```
@kqmlReceivedUnAchieve[atomic]
+!kqml_received(KQMLsenderVar, unachieve, KQMLcontentVar,
   KQMLmsgId) <- .add_annot(KQMLcontentVar, source(
      KQMLsenderVar), KQMLannotG); .drop_desire(KQMLannotG).
```

Recall that the default handling of `achieve` always adds an annotation specifying which was the agent that delegated the goal.

6.3 Example: Contract Net Protocol

This example shows how communication support available in ***Jason*** helps the development of the well known Contract Net Protocol (CNP) [36, 90]. We show an implementation in AgentSpeak of the version of the protocol, as specified by FIPA [45]. In this protocol one agent, called 'initiator', wishes to have some task performed and asks other agents, called 'participants', to bid to perform that task. This asking message is identified by `cfp` (call for proposals). Some of the participants send their proposals or refusals to the initiator. After the deadline has passed, the initiator evaluates the received proposals and selects one agent to perform the task. Figure 6.4 specifies the sequence of messages exchanged between the agents in a CNP interaction.

The project for this example determines that it is composed of six agents: one initiator and five participants.

```
MAS cnp {
    agents:
        c;    // the CNP initiator
        p #3; // three participants able to send proposals
        pr;   // a participant that always refuses
        pn;   // a participant that does not answer
}
```

Participant Code

Three types of agents play the participant role in this example of the use of the CNP. Once started, all participants send a message to the initiator (identified by the name `c`) introducing themselves as 'participants'.

The simpler agent of the application is named `pn` and this agent has only one plan used to introduce itself as a participant. When its initial belief

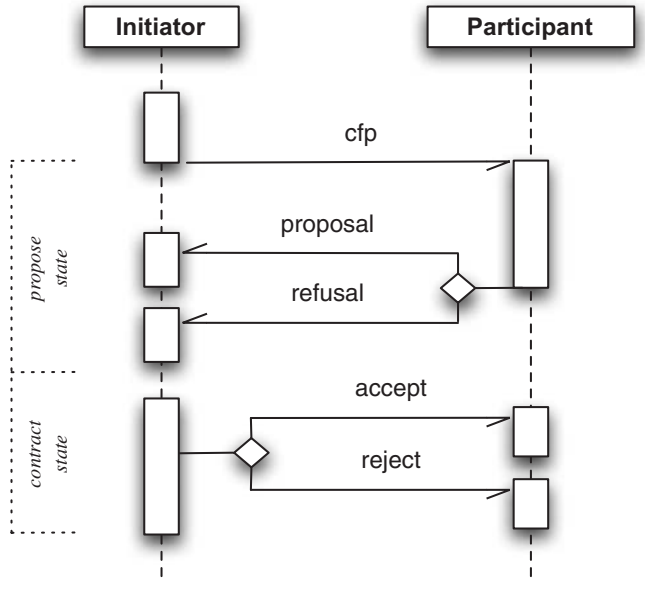

Figure 6.4 Contract net protocol.

`plays(initiator,c)` is added in the belief base, the event `+plays(initiator,c)` is added in the event queue and the respective plan executed. Although it says to the initiator that it is a participant, this agent never sends proposals or refusals.

```
// the name of the agent playing initiator in the CNP
plays(initiator,c).

// send a message to the initiator introducing myself
// as a participant
+plays(initiator,In)
   :  .my_name(Me)
   <- .send(In,tell,introduction(participant,Me)).
```

The second type of participant rejects all calls for proposals. When this agent receives a message from `c` informing it that a CNP is starting, it ignores the content and just sends a refusal message. Each `cfp` message has two arguments: a unique identification of the conversation (an 'instance' of the protocol) so that agents can participate in several simultaneous CNPs, and a description of the task being contracted. In this application, the `cfp` message is sent by the initiator using the `tell` performative; a belief such as `cfp(1,fix(computer))[source(c)]`

is therefore added to the receiver's belief base and the corresponding event `+cfp(1,fix(computer))[source(c)]` added to their event queue. The agent's plan library has a plan, shown below, to react to this event.

```
plays(initiator,c).
+plays(initiator,In)
   :  .my_name(Me)
   <- .send(In,tell,introduction(participant,Me)).

// plan to answer a CFP
+cfp(CNPId,Task)[source(A)]
   :   plays(initiator,A)
   <- .send(A,tell,refuse(CNPId)).
```

Based on the project definition, ***Jason*** creates three instances of the third type of participant (named `p`). The proposals sent by these agents is formed with their price for the task execution. However, in this example, the price is randomly defined by the first rule in the code. This rule is used to answer the `cfp` message in plan `@c1`. The agent also has two plans (`@r1` and `@r2`) to react to the result of the CNP.

```
// gets the price for the product,
// (a random value between 100 and 110).
price(Task,X) :- .random(R) & X = (10*R)+100.

plays(initiator,c).

/* Plans */

// send a message to initiator introducing myself
// as a participant
+plays(initiator,In)
   :  .my_name(Me)
   <- .send(In,tell,introduction(participant,Me)).

// answer a Call For Proposal
@c1 +cfp(CNPId,Task)[source(A)]
   :  plays(initiator,A) & price(Task,Offer)
   <- +proposal(CNPId,Task,Offer); // remember my proposal
      .send(A,tell,propose(CNPId,Offer)).

@r1 +accept_proposal(CNPId)
   :  proposal(CNPId,Task,Offer)
   <- .print("My proposal '",Offer,"' won CNP ",CNPId,
             " for ",Task,"!").
      // do the task and report to initiator
```

```
      @r2 +reject_proposal(CNPId)
         <- .print("I lost CNP ",CNPId, ".");
            -proposal(CNPId,_,_). // clear memory
```

Initiator Code

The behaviour of the initiator agent can be briefly described by the following steps: start the CNP, send a `cfp`, wait for the answers, choose the winner, and announce the result. The complete code of this agent is as follows and is explained below.

```
/* Rules */

all_proposals_received(CNPId) :-
  .count(introduction(participant,_),NP) & // number of participants
  .count(propose(CNPId,_), NO) &      // number of proposes received
  .count(refuse(CNPId), NR) &         // number of refusals received
  NP = NO + NR.

/* Initial goals */

!startCNP(1,fix(computer_123)).

/* Plans */

// start the CNP
+!startCNP(Id,Object)
   <- .wait(2000);  // wait participants introduction
      +cnp_state(Id,propose);   // remember the state of the CNP
      .findall(Name,introduction(participant,Name),LP);
      .print("Sending CFP to ",LP);
      .send(LP,tell,cfp(Id,Object));
      .concat("+!contract(",Id,")",Event);
      // the deadline of the CNP is now + 4 seconds, so
      // the event +!contract(Id) is generated at that time
      .at("now +4 seconds", Event).


// receive proposal
// if all proposal have been received, don't wait for the deadline
@r1 +propose(CNPId,Offer)
   :  cnp_state(CNPId,propose) & all_proposals_received(CNPId)
   <- !contract(CNPId).

// receive refusals
@r2 +refuse(CNPId)
```

```
   :  cnp_state(CNPId,propose) & all_proposals_received(CNPId)
   <- !contract(CNPId).

// this plan needs to be atomic so as not to accept
// proposals or refusals while contracting
@lc1[atomic]
+!contract(CNPId)
   :  cnp_state(CNPId,propose)
   <- -+cnp_state(CNPId,contract);
      .findall(offer(O,A),propose(CNPId,O)[source(A)],L);
      .print("Offers are ",L);
      L \== []; // constraint the plan execution to at least one offer
      .min(L,offer(WOf,WAg)); // sort offers, the first is the best
      .print("Winner is ",WAg," with ",WOf);
      !announce_result(CNPId,L,WAg);
      -+cnp_state(Id,finished).

// nothing todo, the current phase is not 'propose'
@lc2 +!contract(CNPId).

-!contract(CNPId)
   <- .print("CNP ",CNPId," has failed!").

+!announce_result(_,[],_).
// announce to the winner
+!announce_result(CNPId,[offer(O,WAg)|T],WAg)
   <- .send(WAg,tell,accept_proposal(CNPId));
      !announce_result(CNPId,T,WAg).
// announce to others
+!announce_result(CNPId,[offer(O,LAg)|T],WAg)
   <- .send(LAg,tell,reject_proposal(CNPId));
      !announce_result(CNPId,T,WAg).
```

The agent has the initial goal `!startCNP(1,fix(...))`, where `1` means the CNP instance identification and `fix(...)` the task being constructed. The intention created by the `!startCNP` goal is initially suspended for 2 seconds while the participants send their own introductions. When resumed, the intention adds a belief to remember the current state of the protocol: the initial state is `propose`. This belief constrain the receiving of proposal and refusals (see plans `@r1` and `@r2`). Based on the `introduction` beliefs received in the introduction phase, the plan builds a list of all participants' names and send the `cfp` to them. Note that we can inform a *list* of receivers in the `.send` internal action (i.e. `.send` allows for multicast).

The initiator starts to receive proposals and refusals and plans `@r1` and `@r2` check whether all participants have answered and the protocol is still in

the `propose` state. If the initiator has all answers, it can go on and contract the best offer by means of the goal `!contract`. However it could be the case that not all participants answered the `cfp`, and in such case the belief `all_proposals_received(CNPId)` never holds. To deal with this problem, before finishing the intention for `!startCNP`, the agent uses the internal action `.at` to generate the event `+!contract(1)` 4 seconds after the protocol has started. If the initiator receives all answers before 4 seconds, this event is handled by the plan `@lc2`, and so discarded. Note that plan `@lc1` is `atomic`: when it starts executing, no other intention is selected for execution before it finishes. The first action of the plan is to change the protocol state, which is also used in the context of the plan. It ensures that the intention for the goal `!contract` is never performed twice. Without this control, the goal `!contract` could be triggered more than once for the same CNP instance, by plans `@r1`/`@r2` and also by the event generated by the `.at` internal action.

A run of this ***Jason*** application could result in an output as follows:

```
[c]  saying: Sending CFP to [p2,p3,pr,pn,p1]
[c]  saying: Offers are [offer(105.42,p3),
                         offer(104.43,p2),
                         offer(109.80,p1)]
[c]  saying: Winner is p2 with 104.43
[p3] saying: I lost CNP 1.
[p1] saying: I lost CNP 1.
[p2] saying: My proposal '104.43' won CNP 1
             for fix(computer\_123)!
```

6.4 Exercises

1. Considering the domestic robot scenario from Chapter 5:

Basic

(a) Change the AgentSpeak code for the robot so that a message is sent to the supermarket to inform it that a delivery of beer arrived.

Basic

(b) Create a new supermarket agent (called `nsah`) that does not have a plan for goal `+!order(P,Q)`, but once started, uses the performative `askHow` to ask another supermarket for such a plan.

Rewrite the program for the supermarket agent that sent the plan so that after 10 seconds from the start of the application it sends an `untellHow` message to `nsah`.

Advanced

(c) Instead of each supermarket sending the beer price to the robot as soon as it starts (as proposed in the last exercise of Section 3.5), change the code for those agents so that is uses the contract net protocol in such

way that the robot plays the initiator role and the supermarkets play the participant role.

(d) Create a new supermarket agent which, when asked by the robot for a proposal, pretends to be a domestic robot itself and asks proposals from other supermarkets also using the CNP (i.e. it plays the initiator role). With the best offer from the others, this supermarket subtracts 1% from the price and, provided that price still gives the supermarket some profit, sends this new value as its own proposal, thereby probably winning the actual CNP initiated by the domestic robot. Advanced

(e) Create two instances of the previous supermarket agent. This causes a loop in the CNP! Propose a solution for this problem. Advanced

2. Suppose the initiator of a CNP may want to cancel the `cfp`. In order to implement this feature, add a plan in the initiator program for events such as `+!abort(CNPId)` that uses the `untell` performative to inform the participants that the CNP was cancelled. On the participants' side, write plans to handle this `untell` message (this plan basically removes the offer from the belief base). Basic

3. Assume now that participants may also cancel their offer. Write plans (on both sides) to deal with this situation. Note that a participant can cancel its offer only if the protocol state is `propose`. Otherwise the initiator should inform the participant that its proposal was *not* cancelled. Basic

4. Another approach to implement the CNP is to use the `askOne` performative. In this case, the initiator code is as follows (only the plans that need to be changed are shown below). Advanced

```
+!startCNP(Id,Task)
   <- .wait(2000);  // wait for participants' introduction
      .findall(Name,introduction(participant,Name),LP);
      .print("Sending CFP to ",LP);
      !ask_proposal(Id,Object,LP).

/** ask proposals from all agents in the list */

// all participants have been asked, proceed with contract
+!ask_proposal(CNPId,_,[])
   <- !contract(CNPId).

// there is a participant to ask
+!ask_proposal(CNPId,Object,[Ag|T])
   <- .send(Ag,
            askOne,
            cfp(Id,Object,Offer),
```

```
            Answer,
            2000); // timeout = 2 seconds
      !add_proposal(Ag,Answer); // remember this proposal
      !ask_proposal(CNPId,Object,T).

+!add_proposal(Ag,cfp(Id,_,Offer))
   <- +offer(Id,Offer,Ag).  // remember Ag's offer
+!add_proposal(_,timeout).  // timeout, do nothing
+!add_proposal(_,false).    // answer is false, do nothing

+!contract(CNPId) : true
   <- .findall(offer(O,A),offer(CNPId,O,A),L);
      .print("Offers are ",L);
      L \== [];
      .min(L,offer(WOf,WAg));
      .print("Winner is ",WAg," with ",WOf);
      !announce_result(CNPId,L,WAg).
```

The code for the participants also needs changing because no belief is added when receiving an ask message; instead, the required `price` information is directly retrieved from the belief base with a plan for a test goal.

```
// in place of +cfp(....)
+?cfp(CNPId,Object,Offer)
   :  price(Object,Offer)
   <- +proposal(CNPId,Object,Offer). // remember my proposal
```

Compare the two implementations of the CNP and identify advantages and disadvantages of the different approaches.

Advanced

5. Write AgentSpeak plans that agents can use to reach *shared beliefs* under the assumption that the underlying communication infrastructure guarantees the delivery of all messages and that neither agent will crash before messages are received. In ***Jason***, we can say two agents `i` and `j` share a belief `b` whenever `i` believes `b[source(self),source(j)]` and `j` believes `b[source(self),source(i)]`. Typically, one of the agents will have the initiative (i.e. goal) of reaching a shared belief with another agent. Ideally, you should write plans so that the same plans can be used in both the agent taking the initiative as well as its interlocutor who will have to respond to the request to achieve a shared belief.

Advanced

6. When an agent sends an achieve message to another agent, it does not get any feedback. It is thus unable to know whether or when the task has been accomplished. Create a new performative `achievefb` so that the sender will receive a feedback message, as in the following example:

```
!somegoal
    <- .send(bob,achievefb,go(10,20)).
```

```
+achieved(go(X,Y))[source(bob)]
    <- .print("Hooray, Bob finished the job!").
+unachieved(go(X,Y))[source(bob)]
    <- .print("Damn it, Bob didn't do as told.").
```

You could also write plans for this agent to 'put pressure' on Bob, if it is taking too long for Bob to achieve the goal (or the agent realises again the need to have achieved the goal delegated to Bob). Of course, Bob would also need to react appropriately to the pressure (having plans for that), otherwise those messages would just be ignored.

Note that, while the default communication plans are in the `kqmlPlans.asl` file, you can always add plans for communication, similar to the default communication plans, to your agent code (your plans for a particular performative will override the default plans for that performative). Needless to say, if you plan to change the semantics of communication, you should be extra careful when doing this (and ask yourself if it is worth doing); however, in some cases this might be necessary (e.g. if you want your agent to be able to lie). It also worth mentioning that, strictly speaking, we do not need a new performative for the 'feedback' exercise; we just need to make sure the agents implement suitable protocols, so an `achieve` performative leads to confirmation whenever necessary in the given application. On the other hand, the advantage of having two different performatives is that they can be used to make it clear whether the requesting agent expects confirmation or not for a particular request.